1. [25 marks] **Lineup Correction**

   During a morning school assembly, students line up in a single row. The school rule is that students must be arranged in *non-decreasing* order of height (shortest to tallest, ties allowed). Some students are already in the correct position; others are not. Your job is to find **how many students are standing in the wrong position** compared to where they would be in a perfectly sorted lineup.

   You are given an integer array `heights` where `heights[i]` is the height (in centimetres) of the student at position i (0-indexed). Compute the number of students who are not standing in the position they would occupy once the row is fully sorted. Proceed as follows:

   1. Create a copy of `heights` called `sorted_heights`.
   2. Sort `sorted_heights` in non-decreasing order. You **can implement any sorting algorithm of your choice** — bubble sort, selection sort, insertion sort, quick sort etc are all acceptable. **`std::sort` is not allowed in this question.**
   3. Compare `heights[i]` with `sorted_heights[i]` for every index i.
   4. Count and return the number of positions where the two values differ.

   Function signature you must implement:

```
int heightChecker(const std::vector<int>& heights);
```

   The function must accept a `const` reference (do not modify the original array), internally create a sorted copy, and return the count of mismatched positions.

   Worked examples:

```
Input:  heights = {1, 1, 4, 2, 1, 3}
Sorted:           {1, 1, 1, 2, 3, 4}
  [0]: 1 vs 1  ->  same
  [1]: 1 vs 1  ->  same
  [2]: 4 vs 1  ->  DIFFERENT
  [3]: 2 vs 2  ->  same
  [4]: 1 vs 3  ->  DIFFERENT
  [5]: 3 vs 4  ->  DIFFERENT
Output: 3

Input:  heights = {5, 1, 2, 3, 4}
Sorted:           {1, 2, 3, 4, 5}
Output: 5   (every student is out of place)

Input:  heights = {1, 2, 3, 4, 5}
Sorted:           {1, 2, 3, 4, 5}
Output: 0   (already in order, nobody is out of place)
```

   Copy the following `main()` into your file unchanged to test your solution:

```cpp
int main() {
    std::vector<int> h1 = {1, 1, 4, 2, 1, 3};
    std::cout << "Test 1: " << heightChecker(h1)
              << " (expected 3)" << std::endl;

    std::vector<int> h2 = {5, 1, 2, 3, 4};
    std::cout << "Test 2: " << heightChecker(h2)
              << " (expected 5)" << std::endl;

    std::vector<int> h3 = {1, 2, 3, 4, 5};
    std::cout << "Test 3: " << heightChecker(h3)
              << " (expected 0)" << std::endl;

    std::vector<int> h4 = {3, 3, 3, 3};
    std::cout << "Test 4: " << heightChecker(h4)
              << " (expected 0)" << std::endl;

    std::vector<int> h5 = {2, 1};
    std::cout << "Test 5: " << heightChecker(h5)
              << " (expected 2)" << std::endl;

    return 0;
}
```

Expected output:

```
Test 1: 3 (expected 3)
Test 2: 5 (expected 5)
Test 3: 0 (expected 0)
Test 4: 0 (expected 0)
Test 5: 2 (expected 2)
```

Criterion	Marks
Correct function signature; original array not modified	5
Custom sorting algorithm implemented correctly (`std::sort` not used)	8
Correct element-by-element comparison and counter	7
All five test cases produce the expected output	5

2. [35 marks] **Library Book System**

You are building a small catalogue system for a city library. The library holds two kinds of books: *physical books* (which have a page count) and *eBooks* (which have a file size in megabytes). Both share common attributes: title, author, and price. You will model this with inheritance and manage a collection of books polymorphically.

Step 1 — The `Book` base class.

- Declare a class `Book` with the following `protected` data members: `std::string title`, `std::string author`, and `double price`.
- Provide a constructor `Book(std::string title, std::string author, double price)` that initialises all three members. **If `price` is negative**, throw a `std::invalid_argument` with the message `"Price cannot be negative."` *before* assigning the value.
- Declare a pure virtual method: `virtual void display() const = 0;`
- Declare a virtual destructor: `virtual ~Book() = default;`
- Provide public getters: `std::string getTitle() const` and `double getPrice() const`.

Step 2 — The `PhysicalBook` derived class.

- Derive `PhysicalBook` publicly from `Book` and add a private `int pageCount` member.
- Constructor: `PhysicalBook(std::string title, std::string author, double price, int pageCount)`. Call the `Book` constructor from the initialiser list and assign `pageCount`.
- Override `display()` to print the following (use `std::fixed` and `std::setprecision(2)` for the price; include `<iomanip>` at the top of your file):

```
[Physical] "<title>" by <author> | Price: $<price> | Pages: <pageCount>
```

Step 3 — The `EBook` derived class.

- Derive `EBook` publicly from `Book` and add a private `double fileSizeMB` member.
- Constructor: `EBook(std::string title, std::string author, double price, double fileSizeMB)`. Call the `Book` constructor from the initialiser list and assign `fileSizeMB`.
- Override `display()` to print the following (use `std::fixed` and `std::setprecision(2)` for both the price and the file size):

```
[EBook]    "<title>" by <author> | Price: $<price> | Size: <fileSizeMB> MB
```

Step 4 — The `Library` class.

- Declare a class `Library` with a private `std::vector<Book*> books` storing raw pointers to heap-allocated `Book` objects.
- Implement the following public methods:

- `void addBook(Book* b)` — appends `b` to `books`.
- `void removeBook(const std::string& title)` — finds the first entry whose `getTitle()` matches `title` (case-sensitive). If found, `deletes` the pointer, erases it from the vector, and prints `"Removed: <title>"`. If not found, prints `"Not found: <title>"`.
- `void displayAll() const` — calls `display()` on every book in order. If the vector is empty, prints `"Library is empty."`.
- `void displayAbovePrice(double threshold) const` — uses a **lambda** to iterate over `books` and calls `display()` only on books whose `getPrice()` is strictly greater than `threshold`. If none qualify, prints `"No books above $<threshold>."`.
- Destructor `~Library()` — `deletes` every pointer in `books` then clears the vector. This ensures no memory leaks when the `Library` goes out of scope.

Every `Book*` you create with `new` in `main()` must be passed directly to `addBook`. The `Library` destructor then owns and cleans up all memory — do not `delete` any pointer yourself after handing it to the library.

Copy the following `main()` into your file unchanged to test your solution:

```
int main() {
    Library lib;

    lib.addBook(new PhysicalBook("The␣Pragmatic␣Programmer",
                                 "Hunt␣&␣Thomas", 45.99, 352));
    lib.addBook(new EBook("Clean␣Code",
                          "Robert␣Martin", 29.99, 3.5));
    lib.addBook(new PhysicalBook("Introduction␣to␣Algorithms",
                                 "CLRS", 89.99, 1292));
    lib.addBook(new EBook("Design␣Patterns",
                          "Gang␣of␣Four", 39.99, 5.2));
    lib.addBook(new PhysicalBook("The␣Mythical␣Man-Month",
                                 "Fred␣Brooks", 19.99, 336));
    lib.addBook(new EBook("Effective␣Modern␣C++",
                          "Scott␣Meyers", 34.99, 2.8));

    std::cout << "===␣All␣Books␣===" << std::endl;
    lib.displayAll();

    std::cout << "\n===␣Books␣above␣$35.00␣===" << std::endl;
    lib.displayAbovePrice(35.0);

    std::cout << "\n===␣Removing␣'Clean␣Code'␣===" << std::endl;
    lib.removeBook("Clean␣Code");

    std::cout << "\n===␣Removing␣'Nonexistent␣Book'␣===" << std::endl;
    lib.removeBook("Nonexistent␣Book");

    std::cout << "\n===␣Remaining␣Books␣===" << std::endl;
```

```
    lib.displayAll();

    std::cout << "\n=== Testing exception for negative price ===" << std::endl;
    try {
        Book* bad = new PhysicalBook("Bad Book", "No Author", -5.00, 100);
        lib.addBook(bad);
    } catch (const std::invalid_argument& e) {
        std::cout << "Exception caught: " << e.what() << std::endl;
    }

    return 0;
}
```

Expected output:

```
=== All Books ===
[Physical] "The Pragmatic Programmer" by Hunt & Thomas | Price: $45.99 | Pages:
    352
[EBook]    "Clean Code" by Robert Martin | Price: $29.99 | Size: 3.50 MB
[Physical] "Introduction to Algorithms" by CLRS | Price: $89.99 | Pages: 1292
[EBook]    "Design Patterns" by Gang of Four | Price: $39.99 | Size: 5.20 MB
[Physical] "The Mythical Man-Month" by Fred Brooks | Price: $19.99 | Pages: 336
[EBook]    "Effective Modern C++" by Scott Meyers | Price: $34.99 | Size: 2.80
   MB

=== Books above $35.00 ===
[Physical] "The Pragmatic Programmer" by Hunt & Thomas | Price: $45.99 | Pages:
    352
[Physical] "Introduction to Algorithms" by CLRS | Price: $89.99 | Pages: 1292
[EBook]    "Design Patterns" by Gang of Four | Price: $39.99 | Size: 5.20 MB

=== Removing 'Clean Code' ===
Removed: Clean Code

=== Removing 'Nonexistent Book' ===
Not found: Nonexistent Book

=== Remaining Books ===
[Physical] "The Pragmatic Programmer" by Hunt & Thomas | Price: $45.99 | Pages:
    352
[Physical] "Introduction to Algorithms" by CLRS | Price: $89.99 | Pages: 1292
[EBook]    "Design Patterns" by Gang of Four | Price: $39.99 | Size: 5.20 MB
[Physical] "The Mythical Man-Month" by Fred Brooks | Price: $19.99 | Pages: 336
[EBook]    "Effective Modern C++" by Scott Meyers | Price: $34.99 | Size: 2.80
   MB

=== Testing exception for negative price ===
```

```
Exception caught: Price cannot be negative.
```

Criterion	Marks
`Book` base class: protected fields, constructor, pure virtual `display()`, getters	5
Negative-price check in constructor throwing `std::invalid_argument`	4
`PhysicalBook`: correct inheritance, constructor, `display()` format	5
`EBook`: correct inheritance, constructor, `display()` format	5
`Library::addBook` and `displayAll`	4
`Library::removeBook`: find by title, `delete` + erase, correct messages	5
`Library::displayAbovePrice` using a lambda	4
`Library` destructor: all pointers deleted, no memory leaks	3

3. [40 marks] **Generic Undo/Redo System**

Almost every modern application — text editors, graphic tools, spreadsheets — provides Ctrl+Z (undo) and Ctrl+Y (redo). Under the hood, this is implemented with two stacks: one that records actions you have performed (*undo stack*) and one that records actions you have undone (*redo stack*). You will build that system from scratch using templates, a custom resizable stack, and exception handling.

Step 1 — The templated `Stack` class.

Implement a class template `Stack<T>` backed by either a **raw dynamic array** (a `T*` pointer) or a `std::vector<T>` — your choice. If you use a raw array, double its capacity when it is full, copy elements across, and `delete[]` the old array (start with capacity 4). If you use `std::vector`, resizing is handled for you automatically. Do not use `std::stack` anywhere inside your `Stack<T>` implementation.

Provide the following public interface:

- `void push(const T& value)` — adds `value` to the top; resizes if the array is full.
- `void pop()` — removes the top element. Throws `StackUnderflow` (defined below) if the stack is empty.
- `T peek() const` — returns the top element without removing it. Throws `StackUnderflow` if the stack is empty.
- `bool isEmpty() const` — returns `true` if the stack has no elements.
- `T getAt(int index) const` — returns the element at zero-based position `index` from the *bottom* of the stack (index 0 is the bottom-most element). Does not throw and does not modify the stack.
- `int getSize() const` — returns the current number of elements in the stack.
- Destructor — deallocates the internal array with `delete[]`.

Declare a custom exception class `StackUnderflow` that inherits from `std::exception` and overrides `what()` to return the message: `"Stack underflow: cannot pop or peek an empty stack."`

Step 2 — The templated `UndoRedoManager` class.

Implement a class template `UndoRedoManager<T>` that holds two `Stack<T>` objects privately: `undoStack` and `redoStack`.

Provide the following public methods:

- `void performAction(const T& action)` — pushes `action` onto `undoStack` and **clears `redoStack` completely** (call `redoStack.pop()` in a loop while it is not empty). Performing a new action always wipes the redo history.
- `void undo()` — if `undoStack` is not empty, peek the top element, push it onto `redoStack`, pop it from `undoStack`, and print `"Undone: <action>"`. If the undo stack is already empty, print `"Nothing to undo."` without throwing.

- `void redo()` — if `redoStack` is not empty, peek the top element, push it onto `undoStack`, pop it from `redoStack`, and print `"Redone: <action>"`. If the redo stack is already empty, print `"Nothing to redo."` without throwing.
- `void showCurrentAction() const` — if `undoStack` is not empty, prints `"Current action: <top>"`. Otherwise prints `"No current action."`.
- `void showIf(std::function<bool(const T&)> condition) const` — uses `getAt` and `getSize` to loop over all elements in `undoStack` from bottom (index 0) to top, and prints every element for which `condition` returns `true`, each on its own line indented by two spaces: `" <element>"`. If no element satisfies the condition, prints `" (none match)"`.

Copy the following `main()` into your file unchanged to test your solution:

```
int main() {
    // ---- Part A: string actions ----
    std::cout << "=== String UndoRedoManager ===" << std::endl;
    UndoRedoManager<std::string> strManager;

    strManager.performAction("type A");
    strManager.performAction("delete word");
    strManager.performAction("paste text");
    strManager.performAction("type B");
    strManager.performAction("format bold");

    strManager.showCurrentAction();   // format bold

    strManager.undo();                // undo format bold
    strManager.undo();                // undo type B
    strManager.showCurrentAction();   // paste text

    strManager.redo();                // redo type B
    strManager.showCurrentAction();   // type B

    strManager.performAction("insert image"); // new action clears redo stack
    strManager.redo();                // Nothing to redo.

    std::cout << "\n-- Actions containing 'type' --" << std::endl;
    strManager.showIf([](const std::string& s) {
        return s.find("type") != std::string::npos;
    });

    // ---- Part B: integer actions ----
    std::cout << "\n=== Integer UndoRedoManager ===" << std::endl;
    UndoRedoManager<int> intManager;

    intManager.performAction(5);
    intManager.performAction(12);
    intManager.performAction(7);
```

```
    intManager.performAction(20);
    intManager.performAction(3);

    intManager.showCurrentAction();   // 3

    intManager.undo();                // undo 3
    intManager.undo();                // undo 20
    intManager.showCurrentAction();   // 7

    std::cout << "\n-- Integer actions greater than 10 --" << std::endl;
    intManager.showIf([](int val) {
        return val > 10;
    });

    // ---- Part C: exception handling ----
    std::cout << "\n=== Exception Handling ===" << std::endl;
    Stack<std::string> emptyStack;
    try {
        emptyStack.pop();
    } catch (const StackUnderflow& e) {
        std::cout << "Caught exception: " << e.what() << std::endl;
    }
    try {
        std::string top = emptyStack.peek();
    } catch (const StackUnderflow& e) {
        std::cout << "Caught exception: " << e.what() << std::endl;
    }

    return 0;
}
```

Expected output:

```
=== String UndoRedoManager ===
Current action: format bold
Undone: format bold
Undone: type B
Current action: paste text
Redone: type B
Current action: type B
Nothing to redo.

-- Actions containing 'type' --
  type A
  type B

=== Integer UndoRedoManager ===
```

```
Current action: 3
Undone: 3
Undone: 20
Current action: 7

-- Integer actions greater than 10 --
  12

=== Exception Handling ===
Caught exception: Stack underflow: cannot pop or peek an empty stack.
Caught exception: Stack underflow: cannot pop or peek an empty stack.
```

After all operations in Part A, `undoStack` holds from bottom to top: `"type A"`, `"delete word"`, `"paste text"`, `"type B"`, `"insert image"`. Only `"type A"` and `"type B"` contain the substring `"type"`, so only those two are printed by `showIf`.

Criterion	**Marks**
`Stack<T>`: dynamic array, `push` with auto-resize, `pop` / `peek` / `isEmpty` / `getAt` / `getSize` / destructor all correct	10
`StackUnderflow` exception class; thrown correctly by `pop` and `peek`	5
`UndoRedoManager<T>`: `performAction` clears redo stack; `undo` and `redo` move elements correctly	10
`showCurrentAction`: correct output for both empty and non-empty undo stack	5
`showIf`: accepts `std::function` lambda; uses `getAt`/`getSize` to iterate and filter	5
All three parts of `main()` produce exactly the expected output	5